ARCHITECTURE • CONTRACTS • AI DELIVERY

生物图像工具平台
与全 AI 长期迭代技术方案

以稳定的模块边界、统一工具接口和可验证的开发流程，承载 WB、IF、IHC 及后续图像分析工具。

版本：V3.0 方案基线 | 2026-10-13 | 定位：开发蓝图，尚非已实现系统

> **设计结论：**采用本地桌面模块化单体、Python + PySide6、独立计算进程和单仓库分包。平台统一运行机制，工具拥有实验语义；Codex 分别承担规划、实施和独立审查，以任务包和仓库证据交接。

本版根据已补全需求重写，替代前版关于平台定位、技术选型和 AI 开发流程的建议。现有 WB 用作功能与结果参考；本方案不要求沿用其实现。操作系统暂以 Windows x64 为首发默认，中文界面、本地处理、普通二维图像；这是实施假设，不是用户新增确认的约束。

01 需求理解与明确边界
02 技术选型与验证条件
03 系统结构与依赖规则
04 仓库和代码如何分部分
05 统一接入接口
06 任务、进程与故障恢复
07 文件、状态和数据版本
08 图像公共能力与跨工具共享
09 界面架构与交互约定
10 工具接入全流程
11 WB 实现与数值验收
12 IF、IHC 扩展方案
13 全 AI 开发与独立审查
14 跨会话文档与恢复机制
15 自动检查与质量门禁
16 长期演进与重构规则
17 构建、交付和回滚
18 实施顺序与任务拆分
19 可直接使用的任务与会话模板
20 验收清单、风险和依据

01　需求理解与明确边界

核心目标是让代码在工具数量和迭代次数增加后，仍能由新的 AI 会话准确接手、局部修改、验证和回滚。平台是后续工具的共同运行底座。判断设计是否成功，应看接入第二、第三个工具时是否必须反复改平台核心，以及一个陌生会话能否根据仓库材料完成小范围修改。

已确认事项	直接产生的架构决策
以内置模块为主，独立工具经适配器接入	统一桌面外壳与工具 SDK；首期内置工具随平台一起发布，独立可执行程序通过进程适配器接入。
平台统一导航、设置、文件、任务、错误、日志和接口	这些能力进入平台公共层；泳道、实验分组、细胞、染色阈值等不进入平台核心。
WB 可以重写	先提炼可验证行为和参考数据，再按新架构实现；旧代码的偶然行为不自动成为新标准。
后续 IF 强度、IF 计数、IHC 面积	抽取图像读取、坐标、画布等技术能力；每个工具单独管理自己的参数、结果和判定规则。
普通显微视野 TIFF、PNG、JPEG，先二维	先做明确的二维读取与内存预算；预留读取器边界，暂不构建全切片金字塔、三维渲染或集群。
跨工具中间结果共享尚不确定	默认独立运行，允许通过显式导入、版本化文件资产共享掩膜等结果；首期不做自动工作流编排。
Codex 为主，独立会话规划、实施、审查	仓库任务包是交接入口；实施者和审查者使用同一提交及验收标准，审查者重新获取证据。

1.1 平台的首期职责

提供工具发现与导航、公共设置、文件导入导出入口、任务队列、进度与取消、错误分类、日志诊断、结果资产索引、工作区打开保存，以及工具兼容性检查。工作区在这里仅是文件与运行记录的容器，不代表统一实验数据库。

首期不安排账号、角色体系、组织管理、云同步、插件商店、通用低代码表单系统、分布式计算或自动化 AI 代理后台。将来确有需求时再做独立设计。AI 编程流程与产品运行流程分开：产品不依赖 Codex 才能启动和计算。

1.2 四条不可退让的验收原则

1. **局部可理解：**一个工具的算法与测试能够脱离桌面界面运行；新增工具不需要阅读全部工具代码。
2. **规则可执行：**禁止的依赖、错误的数据格式和破坏接口的变更，能在检查中失败。
3. **结果可解释：**每次计算能追溯输入、参数、算法版本和人工修订，失败不会伪装成零结果。

4. **会话可替换**：关键知识保存在版本化仓库与样例里，当前对话结束不影响后续开发。

> “全 AI 编程”不能保证永不出错。这里要建立的是：错误容易暴露、影响范围受限、历史依据保留、修复能够复验的工程体系。

02　技术选型与验证条件

推荐主路线：Python + PySide6 / Qt Widgets，计算放入独立子进程。这个选择基于二维科学图像工具、可重写 WB 和长期 AI 维护优先的组合约束。主要收益是减少界面与算法之间的语言、模型转换和构建链；不是认为单一语言天然不会失控。

候选方案	适配性与代价	本项目结论
Python + PySide6	算法生态与桌面交互同语言；需要严格约束动态类型、Qt 生命周期、线程和打包。	主选。以类型检查、纯算法层、进程协议和早期打包弥补风险。
Tauri + TypeScript + Python	适合 Web UI；同时维护前端、桌面桥接和科学计算运行时，跨边界数据与发布更复杂。	本期不选。未来明确需要 Web 前端复用时重新评估。
Electron + TypeScript + Python	有现有 WB 桌面经验，Web 调试便利；仍需双语言协议与较大的桌面运行包。	保留为原型失败后的备选，不因旧实现存在而强制沿用。
浏览器 + 本地服务	图形开发方便，但增加服务启动、端口、文件访问和生命周期问题。	不满足当前最小运行体系的偏好，暂不引入。

2.1 建议依赖基线

领域	选择	使用边界
语言与依赖	Python；uv workspace + uv.lock	起步候选 Python 3.12；P0 验证依赖轮子、生命周期和构建支持后锁定具体补丁版本，不跟随“最新版”自动漂移。
桌面	PySide6 Qt Widgets	薄视图 + Presenter；画布封装 QGraphicsView。首期不同时引入 QML 或嵌入式 Web UI。
计算与读取	NumPy、SciPy；tifffile、Pillow	按用途引入。scikit-image 在 IF / IHC 需要时加入；深度学习框架不预装为核心依赖。
数据边界	Pydantic v2；标准库 dataclass / Protocol	Pydantic 用于文件与 IPC 校验；算法内部使用明确类型，不把所有数组包装成复杂模型。
持久化	SQLite + JSON + 文件目录	平台只保存通用元数据；工具保存自身配置、修订和结果格式。

领域	选择	使用边界
检查	Ruff、Pyright、pytest、pytest-qt、import-linter	一个主要类型检查器；边界与依赖检查必须在 CI 执行，不仅由编辑器提示。
发布	pyside6-deploy / Nuitka，standalone 目录包	先验证包能启动和运行子进程，再加安装器。锁定构建环境；备用打包路线须通过同一验收。

uv 支持同仓库多个包共用锁文件，但不隔离包的实际 import；因此仍需独立安装测试和依赖边界检查。依据：uv workspaces。部署工具封装 Nuitka，需提前验证原生依赖：Qt 官方部署说明。

2.2 先做技术试验，再冻结选型

P0 只实现一个可打包窗口、一张 16-bit 二维图、ROI 拖动、曲线、后台任务和取消。验证中文路径、125% / 150% / 200% 缩放、无 Python 的 Windows 干净环境以及任务崩溃后界面继续运行。记录依赖版本、启动时间、内存和缺陷。

建议测量样本为 2048×2048 uint16、4096×4096 uint16、4096×4096 RGB，以及 20 张批次；这些是工程试验集，不是产品宣称的最大能力。持续拖动不能因计算卡住，取消必须有可观测状态，打包后的 worker 必须同样可测。原型不通过时，先定位 Qt 适配或打包问题，再决定是否换栈，不带着未知风险建设完整平台。

03 系统结构与依赖规则

桌面装配入口

窗口、导航、工具注册、Qt 进程适配、公共服务注入

平台运行层

任务状态机 · 文件资产 · 设置 · 日志 · SQLite 索引

公共契约层

工具声明 · 任务请求与结果 · 能力端口 · 事件与错误

工具模块

WB / IF 强度 / IF 计数 / IHC 面积

各自包含领域、用例、适配器、界面和测试

图像基础库

读取 · 原始数组 · 坐标 · 技术性几何操作 · 可选公共画布

上图表示职责分组，实际允许的 import 方向以下表为准。平台运行层不会导入具体工具；桌面与 worker 的装配入口负责把两者连接起来。

模块	允许依赖	明确禁止
`tool_contracts`	标准库、Pydantic	Qt、NumPy、SQLite、具体工具、平台实现
`platform_runtime`	tool_contracts、存储与日志依赖	Qt、WB / IF / IHC、实验语义、直接导入工具执行器
`desktop_sdk`	tool_contracts、PySide6	platform_runtime 的内部实现、任何具体工具
`imaging_core`	图像与数值依赖、必要的通用资产类型	Qt、平台实现、工具专用算法
`imaging_ui`	imaging_core、desktop_sdk、PySide6	任意工具的领域类
`tool_x.domain`	本工具纯数据与数学库、imaging_core 的纯技术类型	Qt、数据库、子进程、日志落盘、其他工具
`tool_x.application`	本工具领域、tool_contracts、所需图像公共 API	具体 UI、平台内部、另一工具实现
`tool_x.ui / adapters`	本工具用例与公开数据；UI 可用 desktop_sdk / imaging_ui	反向让领域依赖 UI；跨工具内部调用
桌面 / worker 装配入口	注册已批准的具体实现并注入端口	承载任何计算公式或不断增长的工具特判

3.1 把结构规则变成失败条件

每个包指定唯一公开入口 `api.py`，外部只通过公开入口使用它。import-linter 检查层级、工具独立性与禁用依赖；小型 AST 检查补充“跨包不得 import 内部路径”。动态 import 只允许出现在固定的装配注册器；测试夹具不豁免生产代码的依赖规则。检查同时覆盖间接依赖和测试安装后的真实导入。

协议 SDK 提供窄接口，绝不传递“整个主窗口”“整个应用容器”“数据库连接”或可随意查询的服务定位器。宿主主动注入需要的端口，例如任务提交、资产解析和设置读写。共享事件限定类型与订阅范围，不建立任意字符串的全局消息总线。

import-linter 提供 Python import 契约检查能力；具体规则由本项目编写。依据：import-linter 官方仓库。静态 import 检查无法证明全部运行时行为，仍须集成测试和审查装配代码。

3.2 最容易被 AI 写坏的边界

常见捷径	本方案要求
IF 强度工具直接 import IF 计数的分割函数	共用稳定的技术库，或消费显式的 LabelImage 资产；不得依赖另一工具的内部实现。

常见捷径	本方案要求
主窗口增加 `if tool_id == "wb"` 来处理结果	WB 结果由 WB 的结果视图呈现；平台只处理统一摘要和资产引用。
点击事件中读取图片、计算、写 Excel	事件仅生成用例请求；重操作在 worker，导出由单独用例负责。
不断往 `common/utils.py` 增加函数	先确定语义归属；至少两个真实消费者且语义稳定时才抽取公共能力。

04　仓库和代码如何分部分

采用一个产品仓库管理可兼容的整体版本，以包和导入规则隔离模块。初期不拆成多个 Git 仓库，也不为每个类建立包；需要分别测试、发布依赖或保护边界的部分才独立分包。下列路径是拟建产品仓库的相对路径，不代表已在当前工作区创建。

```
bioimage-workbench/
  AGENTS.md                  # 全局短规则与工作入口
  pyproject.toml / uv.lock   # workspace 与锁定依赖
  dev.py                     # 统一开发命令的薄封装
  apps/
    desktop/                 # Qt 外壳与装配；一个发布应用包
    worker/                  # 无 UI 的 worker 装配入口
  packages/
    tool_contracts/          # 通用数据契约与执行端口
    platform_runtime/        # 调度、状态、文件、设置、日志
    desktop_sdk/             # ToolView 与宿主 UI 端口
    imaging_core/            # 无 UI 的图像基础能力
    imaging_ui/              # 需要复用时引入的画布适配
  tools/
    wb/                      # 首个实际业务工具
    if_intensity/            # 第二工具接入时才创建
    if_count/                # 后续创建
    ihc_area/                # 后续创建
  tests/
    architecture/ contract/ integration/ packaging/
    fixtures/                # 小型、脱敏、来源明确的基准
  docs/
    architecture.md / module-map.md / contracts/
    adr/ / runbooks/ / ai-workflow.md
  work/
    tasks/ / evidence/       # 任务事实与验证证据
  tooling/                   # 检查与生成器，无业务算法
```

每个 packages / tools 成员拥有自己的 `pyproject.toml`、`src/` 和 `tests/`。apps 可先作为一个应用包的两个入口，避免为两个启动脚本重复维护依赖。只在确需独立安装与依赖隔离时再拆成两个应用包。

4.1 单个工具内部固定结构

```
tools/wb/
  AGENTS.md / README.md / pyproject.toml
  src/wb_tool/
    api.py                  # 对外导出，不做初始化副作用
    definition.py           # 静态声明与 operation 列表
    schemas.py              # 参数、结果、草稿与版本
    domain/
      lanes.py              # 泳道身份与布局规则
      profiles.py           # 原始曲线与极性转换
      bands.py              # 候选、边界、人工修订
      quantify.py           # 积分与背景规则
      normalize.py          # 归一化关系与缺失值
    application/
      analyze.py / revise.py / export.py
    adapters/
      executor.py / persistence.py / export_files.py
    ui/
      view.py / presenter.py / view_state.py
  tests/
    unit/ contract/ regression/ ui/
    fixtures/
```

每个文件按一个变化原因划分，而不是按行数平均切割。`view.py` 负责控件，`presenter.py` 负责交互状态，`analyze.py` 编排用例，`quantify.py` 实现数值定义。不能把大文件拆成相互传递巨大字典的多个小文件，然后宣布已模块化。

4.2 一个模块完成必须具备的材料

模块责任说明、公开 API、允许依赖、输入输出样例、关键不变量、对应测试、失败行为、版本迁移规则和本地验证命令。README 只写本模块信息，不复制全仓库规则。公共方法参数及结果有类型；无法消除的第三方 `Any` 收口在适配器，禁止继续传播到领域层。

不预建所有未来工具、不提前做一个“万能分析基类”、不把强度测量和计数强行塞入一个参数模型。使用少量 Protocol 和显式组合；默认不超过必要的继承层。新增文件数量不作为成果指标，可独立验证的行为才是。

05　统一接入接口

统一接口由声明、执行、界面、宿主能力、数据封套五部分组成。这不是必须对外开放的 HTTP API，而是进程内类型接口与进程间 JSON 契约的组合。内置工具的代码可信且随应用发布；进程分离用于稳定性和资源管理，不是恶意插件沙箱。

5.1 五类契约与所有权

契约	必需内容	负责人
`ToolDefinition`	稳定 tool_id、工具版本、SDK / 协议范围、operations、支持的输入资产类型、配置与结果 schema_id、资源声明、帮助入口	工具声明，平台校验
`ToolExecutor`	参数与输入验证；estimate；execute；取消检查；返回类型化结果	工具实现，worker 调用
`ToolView`	创建视图、打开草稿、保存草稿、展示某次结果、询问能否关闭、释放订阅	工具 UI 实现，外壳管理生命周期
`HostPorts / ExecutionContext`	少量类型化端口；UI 侧用于任务 / 文件 / 设置，worker 侧用于只读输入 / 临时输出 / 事件 / 取消	平台提供，工具依赖接口
`RunRequest / RunResult / ArtifactRef`	身份、版本、参数快照、输入来源、结果资产、警告与错误；业务字段放入工具 schema	封套平台所有，载荷工具所有

5.2 注册机制

采用显式内置注册表。桌面入口登记 UI 工厂，worker 入口登记执行器工厂；二者由同一个工具定义清单校验一致性。注册代码可以引用具体工具，平台运行层不可引用。应用启动先读轻量声明，第一次进入工具时再加载 UI，计算时才加载重算法依赖。

工具定义不得从用户项目文件指定任意 Python 模块名。重复 tool_id、缺失 operation、不支持的 SDK 或结果格式在启动 / 接入验证时报告；出错工具显示不可用原因，其余工具仍可进入。导入时不打开窗口、不扫描磁盘、不下载模型、不启动计算。

5.3 请求与结果最小结构

```
{
  "protocol_version": "1.0",
  "run_id": "run-...",
  "attempt_id": "attempt-...",
  "request_id": "request-...",
  "tool_id": "wb",
  "tool_version": "0.1.0",
  "operation_id": "analyze",
  "algorithm_id": "wb.profile_integral",
  "algorithm_version": "1.0.0",
  "input_refs": [{"asset_id": "image-...", "sha256": "..."}],
  "parameters": {
    "schema_id": "wb.analyze_params@1",
    "payload": {"lane_layout_revision": "layout-..."}
  },
  "draft_revision": "draft-...",
  "resource_budget": {"memory_mb": 2048, "cpu_threads": 2}
}
```

示例省略各模型的完整字段及校验器，表示拟议契约，不是可直接运行的 SDK。参数载荷看似 JSON 对象，但必须经过指定版本的工具模型验证，不能任意添加含义不明的键。图像字节不进入此请求。

结果封套字段	含义与约束
run_id / attempt_id	匹配运行与尝试；不能凭“当前打开的图片”接收结果。
result_schema_id	例如 `wb.quant_result@1`；平台索引不拆解其中的泳道字段。
result_ref / artifacts	已验证的结果文件、表格、预览和中间资产引用；较大结果也不通过 IPC 传输。
summary	供任务中心展示的简短标题、完成项数量等通用摘要；不替代完整结果。
quality_flags / warnings	工具定义的质量提示；允许计算成功但结果需要复核。
provenance	输入摘要、参数摘要、算法 / 模型 / 依赖版本、随机种子、耗时与来源修订。

5.4 类型接口示意

```
class ToolExecutor(Protocol):
    def validate(self, request: RunRequest,
                 inputs: InputCatalog) -> ValidationReport: ...
    def estimate(self, request: RunRequest,
                 inputs: InputCatalog) -> ResourceEstimate: ...
    def execute(self, request: RunRequest,
                context: ExecutionContext) -> ToolRunResult: ...

class ExecutionContext(Protocol):
    def resolve_input(self, ref: ArtifactRef) -> ReadOnlyAsset: ...
    def output_path(self, logical_name: str) -> Path: ...
    def emit(self, event: ProgressEvent | WarningEvent) -> None: ...
    def check_cancelled(self) -> None: ...
```

轻量的格式检查可在宿主进行；涉及大文件、图像解码或模型检查的 validate / estimate 也须异步执行。execute 同步运行在 worker 内，领域函数不感知进程通信。工具不得自行启动另一平台任务；用户显式导入其他工具资产后，再提交自己的运行。

5.5 校验和错误约定

传输数据使用 Pydantic 模型生成 JSON Schema；生成的 schema 纳入版本控制供差异审查，不另外手写第二套。数字、布尔、单位、有限值和枚举必须校验；缺失值采用 null + reason，不用 NaN、空字符串和 0 混用。配置、请求和业务载荷默认拒绝未知字段；若要增加字段，必须明确升级 schema / 协议并测试兼容，而非默认旧端能忽略。

统一错误字段为 `code`，`category`，`user_message`，`details_ref`，`retryable`，`run_id`，`field_path`。类别至少包括 validation、unsupported_input、resource、cancelled、execution、storage、compatibility。稳定 code 用于处理，message 用于阅读；底层异常和调用栈进入诊断日志。不能捕获所有异常后返回“分析完成”。

Pydantic 的严格模式可减少隐式类型转换，但 JSON 与 Python 输入的部分规则不同，必须测试实际传输形式：Pydantic Strict Mode。

06　任务、进程与故障恢复

6.1 运行模型

桌面主线程只处理控件与短状态更新。平台运行层拥有状态机与调度策略，通过 ProcessDriver 接口调用桌面中的 QProcess 适配器。每个实际运行默认启动一个短生命周期 worker；第一版默认单任务并发，队列可容纳多个任务。之后根据内存测量增加并发，不简单使用 CPU 核数决定进程数量。

worker 用同版本可执行入口启动；开发态是受控 Python 环境，发布态是已打包的 worker 入口。打包方案可使用独立 worker 可执行文件或同一程序的 `--worker` 模式；P0 决定一种并固定，后者必须在创建 QApplication 和导入 UI 前分流。

Qt 提供异步进程信号；在 GUI 线程使用阻塞等待可能冻结界面。依据：QProcess 官方文档。

6.2 IPC 协议

采用 UTF-8 NDJSON，每行一条完整 JSON 消息，增量缓冲解析，不假设一次 read 就得到一整条消息。设置单帧上限，建议初始 1 MiB；原图、数组和大表格走文件引用。stdout 专用于协议，stderr 用于日志；第三方库意外写 stdout 必须隔离重定向或判为协议错误，不用模糊正则从混杂日志猜结果。

握手消息声明 protocol_version、tool_id、worker_build；启动阶段验证后才发送 run。宿主发送 run / cancel / shutdown，worker 返回 accepted / progress / warning / result / error。每条消息含 run_id、attempt_id、递增 seq；重复、错任务、过期消息不能更新当前任务。进度高频时合并，建议最多每秒 10 次，日志队列有上限。

worker 的控制线程持续读取 stdin，将取消命令写入线程安全标志；计算线程运行同步 execute，并由 check_cancelled 在阶段边界读取标志。输出由单一写入通道串行编码，避免多个线程交错写 JSON。宿主持续异步排空 stdout / stderr，不能等任务结束后才读取。握手设启动超时；控制心跳用于区分进程存活与计算进度，长时间无进度不自动等同崩溃。

request_id 用于重复提交识别：同一工作区中相同 ID 与相同请求摘要返回已有运行，相同 ID 配不同内容则拒绝。worker 的 result 消息只提交 staging 文件清单；宿主发布后才生成最终 ArtifactRef。重试失败尝试可保留 run_id 并新建 attempt_id；已成功运行再次执行应创建新 run，避免覆盖历史结果。

6.3 状态机及取消竞态

```
queued -> running -> succeeded | failed | interrupted
queued -> cancelled
running -> cancel_requested -> cancelled | interrupted

终态一经持久化不再被旧消息覆盖。
retry 生成新的 attempt_id；修改参数则生成新的 run_id。
```

宿主是终态的唯一裁决者。只有结果 schema、资产校验、输出发布和数据库提交全部成功，才进入 succeeded。若成功提交先完成，后来的取消请求返回“已完成”；若 cancel_requested 先被记录，晚到结果不发布为成功。合作取消在阶段和批次边界检查，超时后终止进程；确认已被终止且未发布结果可记 cancelled，无法确定状态则记 interrupted。

默认取消宽限可设 5 秒，重操作允许按 operation 调整并向 UI 显示“正在停止”。超时设置不承诺计算每 5 秒可中断。Windows 中若工具再创建子进程，应由进程适配器统一管理进程树，采用 Job Object 等机制并实测；首版内置执行器禁止自行散落启动不受管理的子进程。

6.4 故障与恢复语义

故障	处理与验收
worker 异常退出 / 崩溃	记录 exit code、stderr 与阶段；任务 failed 或 interrupted，界面保持可用，临时输出不计为成功结果。
应用崩溃后重启	将遗留 running / cancel_requested 转为 interrupted；保留诊断和可验证的已提交结果，不自动宣称“恢复运行”。
磁盘不足 / 输出无权限	预检查加写入阶段检查；失败保留原输入与已有结果；不能留下被索引为成功的半文件。
输入文件在运行前被修改	输入为托管快照，或外部引用在使用前重新核验摘要；不相符则拒绝使用原缓存与旧参数关系。
用户编辑参数时旧任务完成	结果仍关联原 draft_revision；显示为旧修订结果，禁止覆盖当前编辑状态。
内存风险	以解码后体积和中间数组估算准入，限制并发及 BLAS 线程；memory_mb 是预算，未采用 OS 限额前不声称硬限制。
批量部分失败	第一版将每张图像作为子任务，批次汇总成功 / 失败 / 取消数。已完成子任务可保留；重试未完成项，不重新伪造整批成功。

首期仅承诺任务重试和按图像的批次恢复，不承诺任意算法步骤的断点续算。需要检查点时，由对应算法明确定义可持久化阶段及版本，不能仅靠保存 Python 对象恢复。

07　文件、状态和数据版本

7.1 工作区目录

```
workspace/
  workspace.json            # 格式版本、稳定 ID
  platform.sqlite           # 通用索引与任务状态
  inputs/                   # 默认复制的原始输入快照
  drafts/<tool_id>/         # 工具草稿及修订
  runs/<run_id>/
    request.json / provenance.json
    attempts/<attempt_id>/  # 日志、诊断；未发布临时区
    results/                # 校验后不可变结果
  assets/                   # 可复用的掩膜等持久资产
  exports/                  # 用户导出的派生交付文件
  cache/                    # 可删除、可重建缓存
```

默认复制输入到托管目录并计算摘要。将来增加“外部引用模式”时，界面必须说明移动文件会导致缺失，并在重开与运行前核验；重新关联需要内容匹配，不能只按文件名替换。源文件不覆盖，导出不写回输入路径。

7.2 数据库的边界

平台 SQLite 只管理 workspace、runs、attempts、artifacts、tool_instances、namespaced_settings 等通用表及其关系。实验字段保存在工具的版本化 JSON / 数组 / 表格中。工具通过 Repository 端口保存状态，不直接拿数据库连接，也不为每个蛋白或细胞向平台表加列。

一个工作区由一个写入宿主占用，第二实例打开时以只读或明确拒绝处理。单独存储服务串行写库，worker 不写平台数据库。SQLite 工作库放本机目录；跨电脑移动使用已关闭或一致性备份后的完整工作区，不直接把正在写入的数据库放网络共享目录。若启用 WAL，备份走数据库备份机制，不只复制主文件。

7.3 文件与数据库的提交一致性

1. worker 只向本次 attempt 的 staging 区写文件，全部关闭后生成输出清单和摘要。
2. 宿主校验 schema、摘要、路径归属、文件完整性及资产关系；拒绝目录穿越、逃出工作区的链接与路径。
3. 在同一卷内发布到唯一最终目录，然后以数据库事务写入索引与成功终态；临时区和最终区使用唯一 ID，禁止覆盖旧结果。
4. 启动恢复扫描“已发布但未索引”的孤立输出，根据运行记录补偿或隔离；数据库只索引校验完成且存在的输出。

文件系统重命名与 SQLite 事务并非同一个原子事务。上面的顺序和恢复扫描是必需机制；“用了事务就不会半成功”不能作为设计结论。持久化故障应使用故障注入验证。

7.4 必须分别管理的版本

版本	变化触发与责任
app_version / build_id	整个平台发布；记录 Git commit、锁文件摘要、构建环境。
sdk_api_version / protocol_version	工具代码接口 / IPC 语义变化；兼容范围与握手分别检查。
tool_version	工具功能与 UI 变化；不意味着算法必然变化。
algorithm_version	影响结果的公式、默认参数、预处理、排序或阈值策略变化；即使是修 bug 也需明确记录结果影响。
schema_id / workspace_format	工具参数、结果、共享资产与工作区格式分别演进；不可用应用版本代替全部数据版本。
model_version / digest	以后使用模型时单独记录权重、配置、预处理与推理运行时。

旧结果默认只读查看，不因打开项目自动重算。迁移先备份、再转成新副本、最后核验；迁移函数须可重复执行且不重复修改。遇到更高版本格式应明确提示不支持，不能尝试按旧格式覆盖。软件回滚和数据回滚分别设计，升级改变数据后不能只换回旧 EXE。

08　图像公共能力与跨工具共享

8.1 公共图像库只统一技术事实

`imaging_core` 统一像素读取、dtype / shape / axes、图像平面标识、坐标约定、技术性裁剪、掩膜校验、标定信息和数组序列化。WB 的背景扣除、IF 的阳性阈值、IHC 的组织分母属于工具领域；即便代码很像，也不因“都是图像处理”就合并。

对象	明确约定
原始数据	保留读取后的原始数值与 dtype；分析可显式转换 float64，但不可先转成屏幕显示的 8-bit 数据再计算。
二维与多通道	首期接受明确二维灰度、RGB 或已选定的二维多通道平面。不能把 TIFF 的第三轴自动猜成颜色、Z 或时间。
不支持的输入	多页、复杂 OME 轴、异常位深、浮点 TIFF 等按读取器能力明确选择或拒绝；“支持 TIFF”不等于支持所有 TIFF。
坐标	数组索引为 [y, x]；外部点为 (x, y)，原点左上。整数像素索引表示像素位置；矩形数组切片采用 [x0,x1)×[y0,y1)。UI 连续坐标与像素中心的转换只在画布适配器实现并测试。
标定	pixel_size_x / y 与单位可为空；无标定时输出 px / px²，禁止虚构 μm 或 μm²。
显示变换	对比度、伪彩、缩放只影响预览。旋转 / 重采样若参与计算，生成新的处理修订并保存变换、插值方法及来源。
JPEG	可读入并保留来源标识；不能声称其压缩后数值等同原始定量数据。各工具自行决定可接受范围与质量提示。

为每个读取器建立能力表和测试样本。二维 TIFF 可先聚焦 uint8 / uint16；PNG 的灰度位深与透明通道需明确处理；RGB 通道语义不能自动等同 IF 染色通道。超出支持范围的文件返回 unsupported_input，不默默选第一页或做未记录的灰度转换。

8.2 图像资产协议

公共资产首先服务于保存与复用，不创建跨工具共享的可变内存对象。初期定义 `Image2D@1`；真正接入 IF 计数与强度共享时再正式冻结 `LabelImage2D@1` 和 `BinaryMask2D@1`。名称和字段写入 imaging_core 的 schema，不写成平台数据库业务表。

字段组	内容
身份与格式	asset_id、kind、schema_version、内容摘要、媒体格式、文件大小、生产工具及运行 ID。

字段组	内容
图像归属	source_asset_id / sha256、plane_id、geometry_revision、shape、axes；必要时记录来源 ROI 和变换链。
数值语义	dtype、通道选择与名称、像素大小及单位；掩膜的前景定义、标签的 background_id。
生成来源	algorithm_version、参数摘要、模型摘要、人工修订 ID、创建时间、父资产引用。
标签图额外要求	整数且非负，0 为背景，正整数代表对象；不要求 ID 连续。对象计数依据实际对象集合，不能直接使用 max(label)。

二进制 mask / label 可采用无损 TIFF 或禁用 pickle 的 NumPy 数组文件，具体支持格式在 schema 中固定。不要把掩膜保存成 JPEG；不要对标签图使用会产生新标签值的双线性插值。任何网格转换由显式适配用例完成，生成新资产。

8.3 IF 计数 → IF 强度的可扩展示例

1. IF 计数生成对象标签图和对象索引，记录原始图像、平面、几何与分割修订。
2. IF 强度界面允许用户选择“使用已有分割”，通过平台列出兼容资产。
3. 强度工具核验来源图像摘要、shape、平面和几何空间；不一致即拒绝，不能仅因尺寸相同就使用。
4. 强度工具选择测量通道与背景规则，输出逐对象测量表，记录分割资产摘要。
5. 分割被修改后生成新资产。旧强度结果仍保存并可查看；界面提示其基于旧分割，重新运行产生新结果。

计数与强度工具彼此不调用、不修改彼此的数据库、不共享临时变量。自动接力可在将来增加工作流层，但这层依然提交相同的 RunRequest 并连接 ArtifactRef，不侵入算法。

8.4 内存与缓存

普通二维图像仍可能产生大量临时数组。例如 4096^2 的 uint16 原图约 32 MiB，单个 float64 数组约 128 MiB；算法同时保存多个数组时必须计入峰值。默认先逐图计算，限制缓存数量和像素副本。首期 ImageReader 只保证 read_plane；将来真实需要大图时添加区域读取能力，不提前实现空的多分辨率框架。

缓存是可删除的优化。计算缓存键包括输入内容摘要、选定平面 / 通道、几何修订、参数规范化摘要、算法 / 模型 / 数值依赖指纹及随机种子。显示缓存与数值缓存分开。任何影响结果的状态若未纳入键，先关闭该缓存再修设计。参数摘要需稳定序列化、显式展开默认值，排除 NaN / Infinity 并固定数字表示策略。

09　界面架构与交互约定

9.1 公共外壳与工具页面的分工

公共外壳统一工具导航、工作区选择、公共设置、任务中心、错误通知、日志入口、打开 / 保存 / 导出文件对话框和基础视觉规范。工具页面自己组织领域步骤。WB 可以是“泳道 → 条带 → 定量”，IF 可以是“通道 → ROI → 强度”，不强制所有工具套同一个步骤向导。

公共控件只在有稳定复用时抽取：数值参数输入、可撤销 ROI 编辑、图像缩放、结果表、运行按钮及状态、通用文件选择。参数可以统一类型与校验，但复杂工具不靠一份 JSON 自动生成所有 UI。

9.2 三种状态必须分开

状态	责任与保存方式
ViewState	选中的标签页、缩放和焦点等界面状态，允许局部持久化；不参与计算参数摘要。
DraftState	当前可编辑输入、ROI、参数、人工修订；拥有 revision，按工具 schema 保存，支持必要的撤销 / 重做。
RunSnapshot / Result	提交时冻结的输入和参数，以及不可变运行结果；修改草稿不改变已完成运行。

Presenter 接收界面命令，调用应用用例，生成新的状态；View 只渲染状态与发出用户动作。避免控件之间通过信号相互改写参数，最终形成无法追踪的隐式流程。对长期任务的订阅由视图实例持有并在关闭时释放，重复打开不叠加回调。

9.3 操作一致性

运行前显示输入数量、关键参数摘要和必要的校验错误。运行期间显示阶段、已完成数量和取消状态；不能估计完成比例时使用不确定进度，不能编造百分比。关闭工具页可选择保留后台任务；关闭整个应用时明确等待或取消，不能把后台进程遗留到不可管理状态。

人工编辑与自动候选分开保存。人工确认不是覆盖来源记录；每次有效修改增加修订，标注 affected_result 的过期状态。长任务结果只更新对应运行卡片，用户切换文件时不能将旧图结果绘到新图上。

10　工具接入全流程

10.1 新工具的接入协议

1. **写模块卡：**目标、输入、输出、领域语义、资源需求、不支持范围，以及可手工判断对错的小样例。
2. **定义契约：**tool_id、operations、参数 / 结果 schema、资产种类、错误 code 和版本策略。

3. **实现纯计算：**不依赖窗口、数据库和平台单例；先用固定数组完成数值测试。
4. **实现执行适配：**接收 RunRequest，解析资产和参数，调用用例，写 staging 输出，报告事件和取消。
5. **实现工具视图：**通过 HostPorts 选择文件、提交运行、查询状态、保存草稿和展示结果。
6. **填写显式注册：**桌面和 worker 清单均有声明；校验 UI 与执行操作一致。
7. **运行通用接入测试：**错误输入、取消、崩溃、状态恢复、结果校验、无 UI 导入与包独立安装。
8. **加入发布清单：**声明依赖和格式，保存样例、证据、迁移说明及使用帮助。

10.2 完成一次接入应产生哪些变更

通常只修改 `tools/new_tool/`、桌面 / worker 注册清单、依赖锁文件、模块索引和接入测试参数。若必须修改平台核心，先说明缺少哪一种通用能力，并证明它不是新工具的专有需求。公共接口升级走单独兼容变更，不顺手夹在工具接入中。

自动接入测试	可观察结果
声明一致	唯一 tool_id；每个 operation 都有合法 schema、执行器与支持的结果视图。
无 UI 运行	在没有桌面实例、没有 Qt 导入的 worker 路径上完成一个小任务。
输入与参数拒绝	空文件、损坏图像、错资产类型、非有限参数和越界 ROI 返回明确错误，不崩溃宿主。
生命周期	打开 / 关闭工具页多次无重复订阅；取消和完成竞态有单一终态。
产物与重开	结果文件与摘要匹配；应用重开可定位结果，断开的外部引用有诊断。
隔离与资源	worker 被强制终止不影响导航；过大输入被拒绝或排队，日志和 IPC 不无限积压。
独立安装	从构建 wheel 安装到只含声明依赖的环境后测试通过，未借用其他工具的隐含依赖。

10.3 现有独立工具的适配等级

等级	能力与限制
A：原生内置模块	实现全部 SDK 契约；首选新开发路线。
B：命令行进程适配	适配器把任务转换为文件与参数，启动固定程序，读取结构化结果；进度、取消按程序真实能力声明。
C：仅启动外部 GUI	可作为平台快捷入口，但不能宣称已统一任务进度、结果校验和实验可追溯性。需要结果导入器才算进一步接入。

进程适配器使用参数数组和明确工作目录，不拼接 shell 命令。程序路径在配置中显式指定并验证，执行环境和版本写入 provenance。若外部工具只有整段运行，显示不确定进度；不要解析易变的 GUI 文案作为正式接口。

11　WB 实现与数值验收

WB 是第一项业务验证，用来检验平台契约是否足够，同时完成一条真实分析闭环。现有 WB 改进版可以重写；先冻结参考代码与样本，再区分“应保留的结果定义”“已知问题”和“拟改进行为”。旧程序输出不能单独证明科学正确性。

11.1 功能流程与模块归属

步骤	要求	归属
导入图像	显示位深、尺寸、极性和来源；保留原始数值，预览调整不影响定量。	imaging_core + WB 输入适配
建立泳道布局	已知数量、起止范围、宽度及可调整位置；空白泳道保留稳定身份。	wb.domain.lanes
辅助发现	自动给出候选泳道 / 条带和质量提示；允许逐项人工修订，记录来源。	WB 检测用例
曲线与边界	原始曲线、检测曲线、基线及积分范围分别显示；人工边界和基线点持久化。	profiles / bands + WB UI
定量	执行明确的背景及积分公式，输出原始测量和状态；饱和 / 无效区域不伪装为有效信号。	quantify
关联与归一化	显式选择参考关系，检查分母和匹配；无匹配、被排除与真实零值分开。	normalize
复核与导出	结果表、标注图、参数和 provenance；同一 run 的文件相互一致。	export 用例与适配器

延续已有需求背景：Marker 默认可放左侧但可修改；泳道数量和宽度由当前图配置，不能硬编码为某一批扫描；同一 Fig 的泳道身份与顺序通过显式映射保持一致。不要把预设分子量范围作为蛋白识别规则。分子量标定是可选独立能力，需要用户提供 Marker 信息与有效拟合范围；超出范围不能无提示外推。

11.2 先把数值定义写成测试

在当前参考代码 `wb_quant/core.py` 中核验到：每行沿泳道宽度求均值得到 raw_profile；暗条带用 dtype_max 减均值得到 profile；端点之间的线性基线被扣除后，将负差值截到 0，再按 y 方向做梯形积分。**该信号是均值剖面的积分，不再乘泳道宽度。**像素区域面积是另一项指标。

```
p[y] = mean(I[y, x]), x in [x0, x1)
d[y] = dtype_max - p[y]                 # 暗条带约定
b[y] = bL + (bR - bL) * (y-L)/(R-L)
e[y] = max(d[y] - b[y], 0)
signal = Σ(y=L..R-1) (e[y] + e[y+1]) / 2

L、R 为剖面内的闭区间端点，与图像矩形切片半开区间不同。
该差异必须写在类型与转换测试中，不靠开发者记忆。
```

新方案可先实现 `profile_integral@1` 参考模式。若新增二维背景扣除后的像素总和，必须使用另一个 algorithm_id 和明确字段 / 单位，不能悄悄替换 signal 的含义。背景估计、条带检测、定量公式、归一化分别版本化，检测使用平滑曲线不意味着定量也使用平滑值。

基准测试	独立的预期结果
三点已知曲线	d=[10,20,10]，端点基线为 10，e=[0,10,0]，单位采样梯形积分为 10。
加宽但行均值不变	相同剖面的泳道宽度加倍，参考模式 signal 不变；若某次代码变成两倍，说明误引入面积乘数。
平坦背景	完全被基线解释的有效区域得到 0；若没有有效条带，则状态为未检出，不把缺失结果强制写成 0。
显示独立性	改变界面对比度、缩放和伪彩后，同一输入与参数的测量保持相同。
位深与极性	uint8 / uint16 的受控样本验证动态范围和转换；不将 65535 写死为所有图像的上限。
边界与基线	闭区间端点、手工基线、边界越界、单点无效窗与非有限输入都有明确测试。

11.3 归一化要独立建模

现有 `normalize.py` 的一项行为是按位置匹配参考泳道条带并求比值。新系统可以保留此模式，但它不是所有 WB 实验都通用的归一化定义。归一化模型需显式保存 numerator、denominator、匹配规则、排除状态与 reference_id，不能只保存最终一列数字。

如需目标蛋白 / 内参蛋白，再相对对照组归一化，应作为另一个用例：先建立同样本映射与有效测量，再计算每样本比值，最后按声明的对照集合和汇总方法归一。技术重复与生物重复分开标记，不把同一样本多个条带当成独立生物样本。统计分析暂不纳入平台公共核心。

分母缺失、非正、饱和或不可信时返回无效状态与原因，不给无穷大，也不添加一个微小常数掩盖问题。修正匹配或排除条带后生成新的分析修订，保留旧结果和修订记录。

11.4 参考集与改进评估

参考集包含清晰条带、弱条带、相邻双峰、倾斜、黑边、空白泳道、饱和、不同位深及手动改线案例。按来源冻结输入摘要、参数、认可的标注 / 预期值和理由。合成数组验证公式，人工复核样本验证业务意义，旧工具结果用于差异对照；三者不能相互替代。

验收阈值必须在算法实现前或独立复核时确定。精确数组可用接近浮点精度的容差；图像匹配、条带检出和真实测量的容差取决于已标注样本，不在没有数据时承诺“95% 准确率”。任何基准更新都要列出逐例差异、原因和算法版本，不能由实现会话为让测试通过而批量覆盖。

12 IF、IHC 扩展方案

第二个工具优先选择 IF ROI 强度测量，因为它与 WB 的业务语义明显不同，又能复用图像显示和 ROI。通过它检验平台是否过度围绕 WB 设计；再增加计数和 IHC，以实际需求逐步抽取共享能力。

工具	首个可用版本	工具自有领域规则	可复用技术
IF 强度	选图 / 通道 / ROI，计算原始与背景校正的强度，导出表格。	背景区域、测量通道、有效像素、饱和提示、批次可比条件。	读取、坐标、画布、ROI、表格资产。
IF 计数	分割候选、对象筛选、增删 / 合并修订、数量与对象表。	对象定义、边缘对象规则、粘连拆分、面积阈值与排除理由。	标签图、轮廓显示、坐标与人工修订机制。
IHC 阳性面积	组织有效区域、染色分离 / 阈值候选、阳性掩膜、面积比。	染色配置、组织分母、空洞 / 折叠 / 笔迹等排除规则、阈值解释。	RGB 读取、掩膜、面积测量、叠加预览。

12.1 三个不会混淆的数值定义

IF 强度：对有效区域 Ω，像素数 $A=|\Omega|$，积分 $S=\Sigma I$，均值 $M=S/A$。若声明背景均值 B，则校正积分为 $S-A\times B$，校正均值为 $M-B$。默认保留校正后的负值并标明；若业务需要截断，另设显式参数与版本，不能借用 WB 的 max(·,0) 规则。空区域返回无效，不除以零。

IF 计数：计数单位是通过筛选与复核的对象集合。一个对象可能有多个区域，算法必须定义连通性与合并规则；无对象的有效图像可以是 0，分析失败不是 0。标签 ID 是身份，不应在一次编辑后随排序全部重排，除非创建新的标签修订并保留映射。

IHC 阳性面积：默认形式为 Area(positive ∩ valid_tissue) / Area(valid_tissue)。有效组织区域和排除区域必须可视化，分母为 0 返回无效。全图面积比、组织面积比与细胞阳性率是不同指标，不能共用一个无说明的 positive_rate 字段。

上述是拟定的可审查计算约定，最终参数和适用范围需要用实际样本验证。平台只负责让工具的语义被保存和执行，不替实验方法作统一假设。

12.2 模型推理的后续接入

只有基于实际数据证明传统方法不够时，才引入分割模型。模型服务先保持在工具执行器适配边界内；记录权重摘要、许可、输入尺寸、预处理、阈值、推理设备与运行时。下载模型必须有显式入口，计算时不能隐式联网更新权重。

若 PyTorch 等重依赖与主环境冲突，采用独立可执行 worker 环境并沿用相同任务 / 资产协议，且标注兼容矩阵。不要为了单个工具强迫所有轻量工具加载 GPU 运行时。GPU 数值不一定逐位一致，复验使用预先定义的容差并保留设备信息。

13　全 AI 开发与独立审查

开发的基本单元是一项能独立验收的行为变化，而不是“让 AI 完成某整个系统”。规划、实施、审查由独立 Codex 会话承担；合并和发布再核验对应提交的证据。这是产品仓库中的轻量流程，不需要先开发另一个 AI 工作流平台。

13.1 四个环节各自交付什么

环节	输入	交付	约束
规划会话	需求、当前代码与接口、既有测试、已知缺陷	任务包、影响范围、验收样例、拆分和依赖	先调查现状，不根据过时聊天记忆虚构文件和能力。
实施会话	已就绪任务包、明确 base commit	最小实现、对应测试、变更摘要、真实验证证据	只做任务范围；遇到契约问题更新计划，不顺手重构全仓库。
独立审查会话	需求、base / head、diff、相关源文件、基准样例	可复现问题或无阻塞结论、独立执行的检查记录	先独立判断；作者说明是线索，不是事实证明。审查默认不直接改实现。
集成与发布	针对当前 head 的检查、审查结论与迁移说明	通过的合并结果、可安装产物、版本清单和回滚材料	head 发生变化就更新相关证据；不合并未经验证的新修订。

“独立”表示上下文与验证过程分离，不表示换一个聊天窗口便天然避免同类错误。审查必须重新阅读契约、执行关键用例、尝试反例；数值算法还需要与实现不同来源的预期值。

13.2 一个任务的标准循环

1. **定位：**确认正确仓库、工作区、分支、未提交改动和适用指令；保留他人或其他任务的修改。
2. **重现：**缺陷任务先有失败用例；新功能任务先有验收样例。记录基线已有失败。
3. **约束：**列出允许修改的模块、涉及契约、必须保持的不变量与不在范围的事项。
4. **实施：**一次实现一个可观察变化，优先完成一条贯穿接口到结果的最小闭环。
5. **自验：**运行对应模块与反向依赖检查；界面变化验证交互，持久化变化验证重开，发布相关变化验证安装包。

6. **交接：**提交可审查 diff，写明 base / head、命令与退出码、未完成项，不以“应该通过”代替执行结果。
7. **独审：**新会话复核行为、边界、迁移和测试是否有检出能力；提出问题附重现方法。
8. **集成：**解决问题后重新验证；保存合并提交对应证据，更新任务状态与必要文档。

13.3 控制任务大小

普通任务通常只跨一个业务模块及其测试，最多连同一个相邻公共接口的小变更；这是规划提示，不是机械限制。若任务同时包含新 UI、算法替换、存储迁移和 SDK 升级，应拆成有依赖顺序的任务。是否可拆以“每步合并后仍能运行”判断，不以固定 300 行或 500 行判断。

工作树用于隔离并行或交叉进行的修改，不等于必须并行开发。默认一项任务一个短分支；存在多个任务时采用独立 worktree，不能让多个实施会话同时编辑同一工作目录。所有分支以稳定 main 为基线，避免长期维护一个积累未验收功能的 develop 分支。

13.4 失败时怎么继续

连续两次针对同一症状的修复仍无法解释失败，或发现需要越过任务范围时，要求实施会话产出诊断：已观察事实、试过的假设、证据、尚缺信息和最小下一步。重新规划或交给独立审查定位，不继续堆补丁。这个阈值是工作流程建议，可按任务调整。

依赖安装失败、缺失样本、无法在目标系统运行或验收定义冲突时，将任务标记为阻塞并指出具体原因；已完成部分保留可恢复。常规局部实现选择由执行者解决，不为每个命名、文件拆分或可逆操作请求确认。涉及科学计算含义改变、删除历史数据或尚未确定的核心需求时，需要需求决策，而不是 AI 私自猜测。

13.5 避免“AI 自己写代码、自己改答案、自己说通过”

可以由实施 AI 编写测试，但验收的计算定义、关键样例和期望值需先固定或另行独立评估。修改测试预期、放宽阈值、删掉断言、增加 skip / xfail、扩大类型忽略、关闭架构检查或改 CI，必须在 diff 中单独解释，并由审查会话确认没有掩盖问题。不得用一次截图代替后台数值验证。

高风险计算用小规模解析样例、性质测试与真实样本共同验证；必要时故意注入已知错误，例如“积分乘两次宽度”，确认测试确实失败。无需对所有薄适配器做昂贵变异测试，但关键断言应证明能检出目标缺陷。

OpenAI 官方建议把审查规则写得简短具体，把机械检查放入 CI；本方案据此将 AGENTS 指导和自动门禁分工。依据：Customize code review rules for Codex。上面的任务分工与合并策略是本项目建议，不是 Codex 自动提供的保证。

14　跨会话文档与恢复机制

不要尝试把所有历史聊天塞给每一个新会话。保留短而稳定的入口，加上按任务选择的模块材料。文档的目的不是增加文字量，而是让下一会话快速找到可信的契约、当前提交和验证方法。

文件 / 材料	唯一职责
`AGENTS.md`	少量全局规则：仓库入口、依赖边界、真实测试、禁止数据覆盖、如何找到模块指令。不写完整产品需求。
`docs/module-map.md`	模块职责、公开 API 位置、相关测试和维护入口，作为导航索引。
`docs/contracts/`	接口语义、生成 schema、兼容与弃用规则；与代码 / 样例相互校验。
`docs/adr/`	重要决策的背景、选项、取舍、日期和失效条件；更改决策新增替代记录，不抹掉来由。
模块 `AGENTS.md / README.md`	本模块规则、领域不变量、局部命令和样例；避免复制全局内容。
`work/tasks/T-xxx.md`	该任务唯一的范围、状态、验收、执行与下一步记录；关闭后归档。
`work/evidence/T-xxx/`	命令摘要、环境、base / head、退出码、关键输出或报告引用；大日志作为构建产物，任务文件不堆整段日志。
测试 / 样例 / release manifest	运行行为的证据、预期结果的依据和已发布依赖快照。

14.1 信息发生冲突时如何处理

需求与契约定义“应该怎样”，当前代码和测试执行记录“实际上怎样”。发现二者不一致时记录为冲突，不能自动认为现有代码就是正确需求，也不能仅因文档说有某能力就声称已经实现。任务当前状态只在任务文件维护；若另用 Issue 管理，明确一处为状态来源，另一处仅链接。

14.2 新会话固定启动步骤

1. 报告当前仓库位置、分支 / commit 和工作区修改。
2. 读取根指令、模块索引、当前任务，以及目标模块的指令和 README。
3. 读取本次涉及的公开契约、关键实现与相关测试；不用加载所有工具历史。
4. 执行最小基线验证，核实上次交接中的事实。
5. 用几句话复述目标、影响范围、关键不变量和接下来要验证的行为，再开始修改。

Codex 的项目指令发现与启动目录相关，不应假设从仓库根目录启动就自动加载所有后代目录的规则。因此根指令明确要求进入某模块前读取其规则，任务包也列出应读文件；用一次启动验收核对实际生效的指令。依据：Codex AGENTS.md 官方说明。

14.3 会话结束时只交接可继续工作的事实

```
当前任务：T-012
基线 / 当前提交：<base_sha> / <head_sha>
已完成：可观察行为及文件范围
验证：完整命令、环境、退出码、报告路径
尚未完成：具体缺口，不写“基本完成”
当前工作区：是否有未提交修改，属于哪个任务
风险 / 决策：未确认含义、迁移与兼容影响
下一步：一个明确动作和对应验证
不得丢失的约束：与本任务有关的 2-5 条
```

长历史只保留必要结论和证据链接，不无限扩张交接文件。每完成一个里程碑，安排“冷启动验收”：全新会话仅凭仓库入口完成一次小变更并复验。若必须靠原会话口述才能继续，说明项目文档或模块边界还没有合格。

15　自动检查与质量门禁

门禁以缺陷风险为中心，不追求机械的测试数量、100% 行覆盖率或零长度大文件。至少验证数据边界、计算定义、进程失败、持久化一致性和工具独立性；这些是 AI 最容易“看起来写完”但实际上留有系统性缺陷的地方。

15.1 检查分层

检查层	主要检出内容	执行时机
静态与类型	格式、明显错误、公共 API 类型、不可达 / 未处理分支、非法依赖与循环。	每个改动任务、每次合并检查
契约	schema 生成漂移、旧格式读取、未知字段、错误类型、工具声明、公开 API 变更。	修改契约时全工具执行；平时执行涉及模块
领域单元	公式、边界、空输入、极值、饱和、缺失值、单位和不变量。	每个相关改动任务
属性与回归	显示不影响数值、背景变换性质、稳定身份、参考样本差异及随机种子可复验。	相关算法修改；正式发布运行冻结集
运行集成	真实 worker 协议、部分帧、取消竞态、异常退出、磁盘失败、启动恢复、错误输出。	平台 / 执行适配修改与每次候选发布
UI 与交互	参数绑定、旧结果误覆盖、ROI 坐标、关闭订阅、缩放与焦点、基本可访问性。	UI 修改；发布前桌面冒烟
构建与安装	漏依赖、打包 Qt 插件、中文路径、worker 启动、读写目录、干净环境启动。	构建链变化、候选发布

15.2 统一验证入口

下面是需要在 P1 实现的命令约定，当前不是已存在的命令。`dev.py` 只封装已选工具、选择测试范围和汇总结果，不再自建一个通用编排框架。

```
uv sync --locked
uv run python dev.py doctor
uv run python dev.py check --scope wb
uv run python dev.py check --all
uv run python dev.py contract --tool wb
uv run python dev.py package
uv run python dev.py smoke --artifact <bundle_dir>
```

`check --scope` 必须包含该模块反向依赖的必要测试；无法可靠判断影响范围时运行全量。`check --all` 包含静态、契约、全部领域和运行集成检查；打包与目标机冒烟单列并在发布时强制执行。所有命令失败时返回非零退出码，不能因测试列表为空而报告通过。

使用 src 布局，在独立环境中测试构建 wheel，避免工作目录恰好让未打包代码被导入。依据：pytest 集成实践。workspace 的统一开发环境不能替代这种验证。

15.3 合并完成条件

1. 任务验收条件逐条有证据，没有新增未解释失败；已有失败有基线记录，不能冒称全绿。
2. 不存在新增禁用依赖、循环、跨工具内部导入和未审查的 schema 破坏。
3. 受影响的行为、数据、错误路径及恢复路径完成对应验证；文档与实现同步。
4. 独立审查针对明确 head，阻塞问题已解决；新提交使受影响的旧审查结论失效。
5. 任何基准、检查规则、依赖或数据迁移修改都有单独说明。
6. 任务关闭记录可追溯到合并提交；回滚动作明确且保留数据。

有远程仓库时，用受保护分支和必需检查阻止普通路径直接合并；纯本地阶段用统一验证脚本和审查记录执行同样标准。单一执行者若拥有全部仓库与 CI 管理权限，仍可修改门禁，因此这些是工程防错机制，不是不可绕过的安全隔离。应在基础流程验证后限制随意修改质量规则。

15.4 性能验收的诚实口径

在 P0 指定参考硬件并记录 OS、CPU、内存和显卡。初始目标可设为：后台计算期间交互响应 p95 不高于 100 ms；进度更新有上限；取消命令迅速确认，但实际停止时间按算法阶段和宽限区分；峰值内存不超过任务准入预算。以上均是待实测目标，不是已达到指标。

计算耗时按固定数据集报告中位数 / p95，并以基线对比定位回归。性能阈值必须与样本和环境绑定；不能为了通过门禁减少输入尺寸、关闭关键计算或只测空函数。超过目标触发调查，是否阻止发布由明确的适用场景决定。

16　长期演进与重构规则

16.1 新功能放在哪里

判断问题	默认归属
是否涉及所有工具的任务、文件或错误生命周期?	平台公共层，经契约与集成测试验证。
是否是图像坐标、读取或无领域含义的基础运算?	imaging_core；只有一个消费者时可先在工具内部，不急于抽取。
是否涉及蛋白、细胞、染色或实验比较语义?	对应工具领域，不进入平台核心。
两个工具需要相似能力但定义不同?	保留独立实现或抽取更小的技术原语，不强行统一业务规则。
是否需要新的依赖 / 模型运行环境?	先放在工具适配器；冲突时独立 worker，保持统一协议。

16.2 接口升级的三步法

扩展：增加新版本的契约或可选能力，同时保留已支持的读取路径；注册兼容范围和测试。**迁移：**逐个工具迁移，保留旧项目读取 / 转换能力，确认全部消费者更新。**清理：**在明确的破坏性版本中删除旧代码，提供升级和回滚说明。

严格 schema 下“增加可选字段”也可能让旧读取器报错。兼容性由旧生产者 / 新消费者、新生产者 / 支持范围内消费者、旧持久化文件读取等测试矩阵证明，不由 SemVer 数字证明。内置工具随整包升级，首期不承诺任意历史插件都能加载；外部适配器需握手并声明具体支持协议。

16.3 重构与算法改进分开

纯重构的验收是已定义外部行为不变，算法改进的验收是某些行为有明确且被认可的变化。不要同时调整文件结构、数值公式和结果 schema，再只靠新输出做验收。优先沿既有边界局部替换，使用同一输入的并行比较；新旧实现并存必须有移除条件与任务，不能永久堆积 v2 / new / final 文件。

16.4 维护健康指标

指标	观察方式	触发动作
工具新增对核心的影响	除注册与声明依赖外，新增普通工具是否总要改 runtime。	连续出现时审查缺失的公共接口或错误领域下沉。
改动扩散范围	一个局部需求触及多少无关模块，原因是否能解释。	先重划责任边界，再扩功能。

指标	观察方式	触发动作
公共 API / schema 变更	变化频率、迁移成本和消费者清单。	高频变化说明抽象尚未稳定，减慢新增公共能力。
测试可信度	重复失败、经常重跑才过、skip 增长、基准频繁被覆盖。	修复不稳定根因；不以无限重试保持“全绿”。
冷启动可接手性	新会话是否能找到正确模块、重现验证并完成小改动。	修入口、文档和模块 API，不仅增加提示词。
环境可重建性	干净 checkout + 锁文件能否得到同样测试与安装包行为。	修复未锁依赖、隐含文件和环境假设。

每个里程碑做一次维护复盘；出现数据损坏、结果语义误用、反复跨模块修补或重要检查失效时，优先处理原因再继续添加同类功能。不用“代码超过几万行”作为启动整仓库重写的理由。

17　构建、交付和回滚

17.1 可复现的构建入口

开发环境和构建环境分开。CI 使用锁定 Python、依赖锁、编译器与部署配置；只从指定提交和干净工作树构建。输出 release manifest：app / tool / algorithm 版本、Git SHA、锁文件摘要、协议与格式范围、包含工具、构建平台和产物校验值。

第一版优先 standalone 目录包，先确保 Qt 平台插件、图像解码依赖、数值原生库与 worker 都存在，再增加安装器。不要因为单文件 EXE 看起来方便就接受首次启动解压、杀毒误报或子进程定位问题。选择单文件模式必须重新验证同样场景。

17.2 交付验收

在没有开发工具和 Python 的目标 Windows 环境执行：启动、打开测试图、修改 ROI、完成 WB 最小任务、取消任务、关闭重开、导出文件、查看日志。覆盖中文用户名 / 路径、只读安装目录、不同 DPI、移动工作区和断开的外部输入。运行数据写入工作区或用户可写目录，不写安装目录。

候选包通过冒烟后保存不可变版本；更新采用安装新版本并迁移数据的显式流程，暂不开发复杂后台自动更新器。发布前检查所用 Qt 模块及其他依赖的分发许可与通知文件，具体要求按实际选用模块确认。依据：Qt for Python 许可说明。

17.3 回滚不是仅撤销提交

代码回滚通过 revert 或切回已验证的安装包；数据格式迁移前保留可恢复副本。迁移失败时不能修改原项目到一半；新格式项目若旧程序不支持，应使用迁移前副本或只读导出，不强行降级。每次涉及格式变更的发布都做一次真实恢复演练，并记录是否丢失迁移后的新操作。

源文件和已完成结果按保留策略清理，缓存可自动淘汰。删除结果与工作区是显式操作；回滚脚本不得通过广泛递归删除“恢复环境”。诊断包默认只包含版本、错误和运行元信息，是否包含原始图像由显式导出选择控制。

18　实施顺序与任务拆分

优先建立一条短闭环，再扩大功能。每个里程碑都有可演示产物和退出条件；进度以这些条件判断，不在缺少样本与基准时给出精确工期。下表是规划粒度，具体实施会话仍应使用任务包。

阶段	交付	退出条件
P0 选型验证	窗口、二维图像 / ROI、曲线、worker、打包原型。	干净目标机通过启动 / 计算 / 取消；有内存与 DPI 测量，冻结栈和版本。
P1 仓库底座	包结构、开发命令、类型 / 依赖门禁、任务模板、根指令。	干净 checkout 可重建；故意加跨层依赖会失败，新会话能接手。
P2 平台最小闭环	契约、注册、参考工具、任务状态机、文件输出、重开与日志。	真实 worker 成功 / 失败 / 取消 / 崩溃恢复测试通过。
P3 WB 最小闭环	单图、手工泳道、原始曲线、手工边界、积分、保存与导出。	解析样例和冻结参考样本通过，界面编辑不改变未相关数值。
P4 第二工具验证	IF ROI 强度工具；只复用确实需要的画布 / 读取。	不加入平台实验特判即可接入；边界、接口和数据独立性通过。
P5 业务完善	WB 候选与批量、归一化；IF 计数；IHC 面积，分别分批开发。	每个工具的真实样本、修订、质量提示与导出通过独立验收。
P6 稳定发布	兼容迁移、性能、安装包、备份恢复、维护手册。	全量门禁、干净机冒烟、冷启动会话和恢复演练通过。

18.1 首批任务队列

编号	单项行为目标	依赖	关键验收
T-001	冻结现有 WB 参考和小型样本集	无	来源、摘要、结果定义、已知差异可追溯。
T-002	Qt + worker + 打包技术试验	无	目标机运行、取消、DPI、内存记录。
T-003	仓库、依赖锁、检查入口与 AI 任务约定	T-002	新会话从干净环境完成检查。
T-004	通用契约 v1 与无 UI 参考执行器	T-003	严格数据校验与 schema 快照。

编号	单项行为目标	依赖	关键验收
T-005	只读输入快照与版本化资产引用	T-004	修改来源、错误路径、摘要不一致被发现。
T-006	真实进程协议和任务状态机	T-004	分帧、错序、取消、崩溃、超时测试。
T-007	结果发布、SQLite 索引与重启恢复	T-005/006	写文件和提交之间故障可恢复。
T-008	外壳、导航、设置、任务中心与参考工具	T-004/006/007	工具生命周期、日志、无重复订阅。
T-009	二维读取、坐标与 WB 图像视图	T-005/008	位深、坐标、缩放和显示独立性。
T-010	WB 参考积分的纯计算实现	T-001/004	解析样例、宽度性质与边界测试。
T-011	WB 手工标注到结果的闭环	T-007/009/010	保存重开、编辑修订、独立导出。
T-012	IF ROI 强度接入与共享能力提炼	T-011	第二工具不侵入平台核心。
T-013	WB 自动候选、修订与质量提示	T-011/012	冻结样本与手动回退，结果定义未漂移。
T-014	WB 归一化与批次任务	T-013	空白泳道、匹配失败、分母无效、部分失败。
T-015	IF 计数和标签资产	T-012	对象身份、人工修订、标签格式。
T-016	IF 强度消费既有标签	T-015	拒绝错图 / 错几何 / 过期资产。
T-017	IHC 阳性面积接入	T-012	分母可视化、排除规则、空组织处理。
T-018	候选发布、恢复演练与维护验收	本次发布范围	安装包、迁移、冷启动与全部门禁。

T-008、T-013、T-014、T-017 是功能组，规划会话应进一步拆成数个可以独立合并的任务；不能把表中一行自动理解成一个巨大的实施会话。每次只推进就绪的下一项。允许独立任务并行，但契约和注册表等共享位置由一项集成任务统一修改。

18.2 “平台已完成”的准确含义

达到 P4 后，如果参考工具、WB 最小闭环和 IF 强度都通过统一接口运行，且新增工具无需改动平台业务逻辑，便可认为平台首期底座通过验收。平台不需要等所有生物工具完成才算可用；也不能只有导航壳和几个按钮就宣布完成。

19　可直接使用的任务与会话模板

本节是未来产品仓库的模板。尖括号内容应在实际使用时替换；命令需由 P1 实现并验证后使用，不能把模板中的 PASS 当成真实检查记录。

19.1 模块卡模板

```
模块：<稳定名称>
责任：<一句话说明处理何种变化>
不负责：<容易越界的相邻职责>
公开入口：<api.py / schema / operation>
允许依赖：<精确包或公开 API>
禁止依赖：<反向层、其他工具、UI 等>
输入与输出：<类型、单位、空值与版本>
不变量：<必须保持的行为>
失败行为：<错误 code、是否可重试>
验证：<测试、样例及命令>
变更影响：<反向依赖与历史数据>
```

19.2 单项任务包示例

```
ID：T-010-A
标题：实现 WB 均值剖面参考积分
状态：ready
基线：<base_sha>

目标
  给定已验证的剖面与闭区间边界，计算线性基线以上的
  正差梯形积分，与 reference mode 的定义一致。

不在范围
  不改条带检测、归一化、UI、任务协议或数据库。
  不引入二维像素总和方法。

允许修改
  tools/wb/src/wb_tool/domain/quantify.py
  tools/wb/tests/unit/test_quantify.py
  对应数值定义文档及本任务记录。

输入 / 输出
  输入：有限浮点剖面、L/R、可选基线端点；单位明确。
  输出：signal；错误由 WB 领域错误类型表示。

不变量
  定量使用未平滑剖面；不乘泳道宽度。
  L/R 是闭区间端点；至少两个采样点。
  不修改输入数组，不读写文件，不导入 Qt。

验收
  d=[10,20,10]、基线两端均为 10，signal=10。
```

```
  相同剖面、不同泳道宽度，signal 相同。
  平坦有效剖面 signal=0；非有限值 / 越界明确报错。
  容差按浮点解析样例设定并写明，不能为迁就输出放宽。

验证命令
  <本仓库已实现的对应模块检查命令>

停止扩展的条件
  若需要改变请求 schema 或积分含义，记录冲突并重新规划。

回滚
  撤销该纯函数及调用方提交，无持久数据迁移。

实施证据
  <head_sha、命令、环境、退出码、报告路径>
独立审查
  <审查 head、问题或结论、复验记录>
```

19.3 根 AGENTS.md 建议内容

```
# 项目执行规则
1. 先确认仓库、分支、未提交改动和当前任务；不得覆盖其他工作。
2. 阅读 docs/module-map.md，以及目标模块的 AGENTS.md / README。
3. 按任务范围实施；不要附带全仓库重构或引入未说明依赖。
4. 外部只使用模块公开 API；工具之间禁止导入内部实现。
5. 领域层不得依赖 Qt、数据库、子进程或窗口对象。
6. 原始输入和已完成结果不可覆盖；改变计算含义必须更新版本。
7. 用真实检查结果报告状态；没有运行的检查写“未运行”及原因。
8. 预期值、容差、skip、类型忽略与门禁变化必须单独说明。
9. 文档、schema、测试和实现矛盾时先记录并解决，不能静默选边。
10. 收尾记录提交、证据、未完成项和下一步，更新唯一任务状态。

# 审查重点
- 识别显示数据是否误入定量路径。
- 识别错误 / 缺失是否被转换为成功或零值。
- 识别取消竞态、旧结果覆盖新草稿、输出半提交。
- 识别平台是否出现具体工具的领域特判。
```

这些规则只是建议底稿，建立仓库时应压缩成实际需要的内容。模块指令补充自己的公式与边界；不要把本技术方案全文复制成 AGENTS.md，也不要为了增加上下文上限而持续堆砌历史。

19.4 规划会话提示词

```
请为任务 <需求> 进行规划，先调查当前仓库，不修改产品实现。
读取项目规则、模块索引、相关公开契约、实现和测试。
区分已实现事实、待验证假设和需要需求决策的内容。
输出 work/tasks/<ID>.md：目标、不在范围、base commit、
允许修改范围、依赖、接口影响、不变量、验收样例、验证命令、
回滚和停止扩展条件。
每个实施任务必须有独立可观察结果；过大时按可合并步骤拆分。
对常规实现选择给出决定，不反复询问；核心语义不清时集中列出。
不要提前生成整个系统，也不要声称尚未运行的能力已经可用。
```

19.5 实施会话提示词

```
按 work/tasks/<ID>.md 实施当前任务。
先确认仓库、base commit、未提交修改和相关模块规则，复述本次范围。
先重现缺陷或建立验收样例，再完成最小实现。
保持公开契约和不变量；不能通过削弱测试、放宽容差、增加忽略
或跳过检查来宣称完成。必要的契约变化先记录并调整任务计划。
运行对应验证与反向依赖检查，记录真实命令、环境、退出码和 head。
结束时说明可观察变化、证据、未完成项及下一步，更新任务记录。
遇到相同失败反复修补仍无解释时，转为诊断，不继续叠加补丁。
```

19.6 独立审查会话提示词

```
请独立审查任务 <ID>，比较 base=<SHA> 与 head=<SHA>。
先阅读任务、相关契约和样例，再检查 diff 与必要上下文。
作者结论不是验证证据；请自行复现关键路径并尝试反例。
重点检查：依赖边界、数值定义、缺失值、错误处理、取消竞态、
状态修订、文件发布、兼容迁移，以及测试是否真正能发现目标问题。
特别说明测试预期、门禁、依赖锁和 schema 是否被修改及其合理性。
默认不直接修改实现。只报告可行动问题，附位置、触发条件、
后果和重现方法；区分阻塞缺陷与非阻塞建议。
如无阻塞问题，列出你实际验证的范围及仍未覆盖的限制。
结论必须绑定 head；任何后续实现改动需要相应复审。
```

19.7 发布核验提示词

```
对候选发布 <版本 / commit> 做集成核验。
确认当前提交与审查及 CI 证据一致，工作树和依赖锁状态明确。
执行全量检查、构建、目标机冒烟及涉及的迁移 / 恢复演练。
生成 release manifest、已知限制和回滚说明。
缺失证据时标记未完成；不要把源码测试通过等同于安装包可用。
不要覆盖旧安装包、原始输入或迁移前备份。
```

20　验收清单、风险和依据

20.1 进入长期迭代前的验收清单

编号	必须证明的能力	证据形式
A-01	平台核心不存在 WB / IF / IHC 领域类型与特判。	依赖图、代码搜索、架构门禁。
A-02	WB 与 IF 强度按同一契约接入，工具之间无实现依赖。	两条真实闭环、接入测试与 diff。
A-03	无 UI 环境可以调用工具算法与执行器。	独立安装、headless 测试。
A-04	任务失败、取消、崩溃与重启不会损坏输入或伪造成功。	真实进程与持久化故障注入。
A-05	原始像素、显示参数和计算修订分离。	性质测试、UI 操作与结果比对。
A-06	结果追溯到输入、参数、算法和人工修订。	运行快照、资产摘要和导出检查。
A-07	新增工具只改本模块、注册和声明依赖等合理位置。	第二工具接入审查记录。
A-08	旧工作区按支持矩阵读取 / 迁移，新格式不会被旧程序覆盖。	冻结历史样例、迁移与回滚测试。
A-09	新会话不用原聊天也能完成小改动。	冷启动任务与验证证据。
A-10	干净目标机上的安装包可完成同样业务闭环。	产物摘要、目标机冒烟记录。

20.2 主要风险与处理时点

风险	处理方法	最晚时点
Qt / 原生依赖打包困难	P0 实际打包，保留版本化部署配置和备选路线，不等到功能完成再试。	选型冻结前
平台过早抽象为庞大框架	参考工具 + WB + IF 验证共性；无真实消费者的扩展点不实现。	P4 前复盘
AI 模块边界逐渐失效	类型 / import / schema 门禁，任务范围控制，跨模块变更独立规划。	每次合并
测试与实现一起发生同类错误	先定义预期、解析数组、性质验证、独立审查与已知错误注入。	算法首版验收前
数值正确但实验含义不适用	明确方法适用范围，用真实样本和领域规则验证；保留质量提示和手工复核。	各工具宣称可用前
历史结果因升级失去含义	独立算法 / schema 版本、不可变快照、旧结果只读、迁移副本。	首次保存格式前

风险	处理方法	最晚时点
将大图 / 重模型硬塞主进程	输入准入、资源预算、独立进程；达到真实需求后再扩读取器或运行环境。	新能力接入前

20.3 决策依据与事实范围

本方案的架构分层、接口字段、状态机、任务规范、阶段划分和初始性能目标属于结合已确认需求提出的工程设计，不声称来自某个官方标准或已在此产品中实现。外部资料用于核验工具能力与限制，最终开发仍需锁定依赖并通过原型验证。

依据	对应内容
Qt for Python	Python 桌面路线及 Qt 官方绑定；具体选型取舍为本方案判断。
QProcess	进程生命周期、异步输出与避免 GUI 阻塞等待。
uv workspaces	分包、单锁文件，以及 Python workspace 不提供运行时依赖隔离的限制。
Pydantic 严格模式	边界校验与隐式转换限制；仍需自行定义单位和业务语义。
import-linter	可执行的 import 契约；规则与补充测试由项目实现。
pytest 集成实践	src 布局与安装产物测试，减少环境掩盖问题。
Codex 项目指令	指令层级与发现行为；不能用大篇幅指令代替工程检查。
Codex 审查规则	具体、简短的审查规则，以及机械检查与判断的分工。
Qt 部署工具	部署工具和构建链；发布方案需在目标环境实测。

本地 WB 参考：`C:\Users\70236\ResearchVault\Codex-Workbench\wbquant-improvements\wb_quant\core.py`、`normalize.py`、`io.py`。这些文件可继续变化；正式开发时 T-001 必须冻结具体提交或文件摘要，而非长期依赖“当前文件”。此处对现有算法的描述仅覆盖已核验的剖面、积分、归一化和输入处理，不代表完整正确性审计。

> **建议的实际起点：**先执行 T-001 和 T-002，得到冻结的参考定义与可运行的桌面 / worker 打包原型；通过后按 P1、P2 建立最小底座。不要先让 AI 一次性生成全部平台和所有工具。

交付说明：本文是技术方案与可复用开发模板。接口名称、目录、任务与命令为拟议设计；尚未创建新产品仓库、实现 SDK、执行产品测试或发布安装包。文档制作过程中仅核验资料、整理方案并检查文档呈现。